

- P1. Grupo

Número de grupo: 1

Integrantes:

- Tomas Lucero K.
- Santiago Avalloni G.
- Benjamín Gallardo.
- Jose Morales

Todos los integrantes somos de la sección 2

- P2. Hito 0

Fuente de datos.

Base de datos: [CS:GO Competitive Matchmaking Data](#)

Entidades (alguna de ellas): id_game, id_equipos, data_partida, resolucion_partida.

- El Problema

Motivación: Entender el comportamiento de los jugadores profesionales de csgo y ver como las acciones tienen impacto en la partida.

Ejemplos: Donde se lanzan las utilidades y que enemigos y/o aliados se ven afectados

- tiempo en el que se planta la bomba
- tipo de ronda (echo, semi echo, full build)
- economía de la ronda.

- Estudio de datos (a priori)

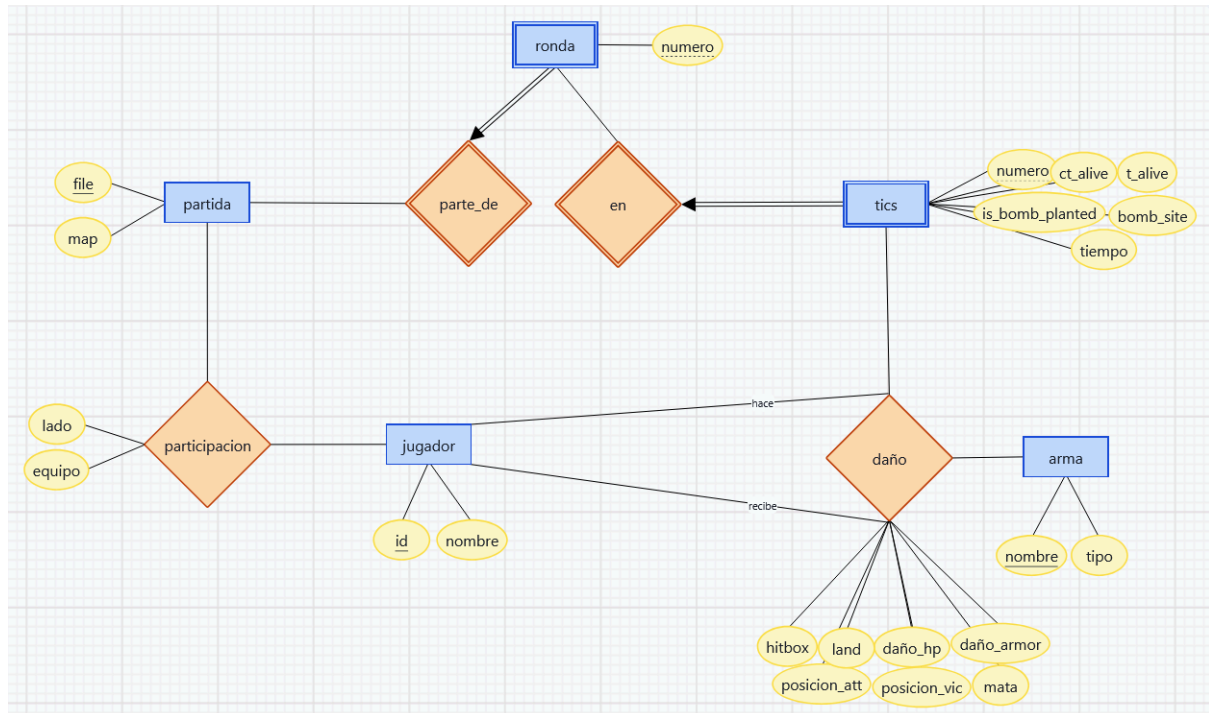
pd: Si el lector no conoce el cs:go, aquí dejo la definición según google:

[Counter-Strike](#) (CS) es una saga de videojuegos de disparos tácticos en primera persona (FPS) multijugador, desarrollada por Valve.

Los jugadores se dividen en equipos de terroristas y antiterroristas, compitiendo en rondas basadas en objetivos como desactivar bombas o rescatar rehenes. Es conocido por su alta competitividad y su impacto en los esports.

- Jugabilidad: El juego se basa en el trabajo en equipo, la estrategia y la gestión de recursos. Tras cada ronda, los jugadores reciben dinero según su desempeño (ganar, perder, eliminar enemigos) para comprar armas más potentes.

- P3. Modelo Entidad Relación



Las entidades creadas son **Jugador, Partida, Arma, Ronda y Ticks**, las tres primeras son lo que podríamos considerar entidades básicas en un juego mientras que ronda y ticks son las subdivisiones de partida y ronda respectivamente por lo que si es que se necesita revisar el momento más exacto en el que ocurrió cierto evento es necesario tener esta entidad más granular. En **Ticks** tenemos muchos atributos ya que en cada momento cada uno de estos puede cambiar siendo un tick la unidad de tiempo dentro del sistema mas pequeña que se puede tener

Las restricciones que no pueden ser visibles del modelo ER que se presenta aquí son, primero, que en el conjunto de datos que encontramos hay solo algunos jugadores que tienen un equipo con nombre mientras que al resto se les tiene en un equipo genérico "Team 1" o "Team 2"; segundo, en los datos que tenemos existe la anomalía de borrado ya que los datos de jugador y ticks provienen de una gran tabla daño (de los datos originales) por lo que si es que en algún tick un jugador no hizo daño no se tiene la información de los atributos de la entidad que nosotros catalogamos como ticks y si es que un jugador no hizo daño y no recibió daño no va a aparecer directamente en la tabla jugador (solo aparecería en el conteo de jugadores en la entidad ticks)

Por esta anomalía ya presente en los datos que encontramos tendremos que tener cuidado cuando queramos analizar la información que tenemos ya que esta no es exhaustiva.

- P4. Modelo Relacional

Entidades:

- **Partida**(file*:str, map:str)

- **Ronda**(file*:str, numero**: int)
file REF Partida(file)
- **Ticks**(file*:str, numero_ronda*:int, numero**:int, ct_alive:int, t_alive:int, is_bomb_planted:bool, bomb_site:[A, B, None], tiempo: time/date)
file, numero_ronda REF Ronda(file, numero)
- **Jugador**(id*:int, nombre:str)
- **Arma**(nombre*:str, tipo:str) {nombre*} -> {tipo}

Relaciones:

- **Participación**(file*:str, id_jugador*:int, lado:bool, equipo:str)
file REF Partida(file), id_jugador REF Jugador(id)
- **Daño**(att*:int, vic*:int, arma*:str, partida*:str, ronda*:int, tick*:int, land:coordenadas, hitbox:bool, daño_hp:int, daño_armor:int, posicion_att:coordenadas, posicion_vic:coordenadas, mata:bool)
att, vic REF Jugador(id, id)
arma REF Arma(nombre)
partida, ronda, tick REF Ticks(file, numero_ronda, numero)

- P5.

5. Explicación y justificación de Formas Normales

El modelo relacional propuesto se encuentra en Forma Normal de Boyce-Codd (BCNF o FNBC) y, por consiguiente, también cumple con la Tercera Forma Normal (3FN).

1. Partida:

- Dependencia: {file} -> {map}
- Justificación: {file} es la llave primaria y define únicamente cada tupla. Es superllave. Cumple FNBC.

2. Ronda:

- Dependencia: La llave compuesta {file, numero} define la existencia de la ronda.
- Justificación: No hay atributos descriptivos que generen dependencias problemáticas. Cumple FNBC.

3. Ticks:

- Dependencia: {file, numero_ronda, numero} -> {ct_alive, t_alive, is_bomb_planted, bomb_site, tiempo}
- Justificación: Para saber el estado exacto del mapa y los jugadores, se necesita la llave compuesta completa. Como esta llave define únicamente el estado del tick, es superllave. Cumple FNBC.

4. Jugador:

- Dependencia: {id} -> {nombre}
- Justificación: {id} define de manera única al jugador, siendo una superllave. Cumple FNBC.

5. Arma:

- Dependencia: {nombre} -> {tipo} (Entregada explícitamente en el enunciado).
- Justificación: Dado que {nombre} es la llave primaria destacada por el desarrollador (llave candidata y, por tanto, superllave minimal), define únicamente cada tupla. Cumple FNBC.

6. Participación:

- Dependencia: {file, id_jugador} -> {lado, equipo}
- Justificación: Un jugador puede cambiar de equipo o lado dependiendo de la partida, por lo que el atributo {equipo} o {lado} depende de la combinación completa de la partida y el jugador. Al depender de la llave compuesta completa, el determinante es superllave. Cumple FNBC.

7. Daño:

- Dependencia: {att, vic, arma, partida, ronda, número} -> {land, hitbox, daño_hp, daño_armor, posicion_att, posicion_vic, mata}
- Justificación: Para registrar un evento único de daño sin ambigüedades, se requiere identificar al atacante, la víctima, el arma, y el momento exacto (partida, ronda, tick/número). Toda esta llave primaria compuesta gigantesca es necesaria para determinar de forma única los atributos descriptivos como el daño infligido o las posiciones. Como este conjunto define únicamente la tupla del evento, es una superllave. Cumple FNBC.